

DUOC UC · DESARROLLO BACKEND III (PBY2203)

Guion del video

Modernización del Backend del Banco XYZ

Qué decir, qué mostrar y qué ejecutar en cada momento · 5 a 7 minutos

Estudiante	[Nombre del estudiante]
Asignatura	Desarrollo Backend III
Evaluación	Evaluación Final Transversal (Semana 9)
Repositorio	github.com/camiluck/banco-xyz-eft
Fecha	Octubre 2026

Duración: 5 a 7 minutos (este guion dura unos 6:30 leyendo a ritmo normal) · **Formato:** MP4 · **Webcam visible todo el video**

Antes de grabar (15 minutos antes)

1. Abre **Docker Desktop** y espera a que diga *Engine running*.
2. En una terminal, dentro de la carpeta del proyecto:

```
docker compose up -d --build
docker compose ps          # espera a que TODOS digan (healthy), 3 a 5 minutos
./scripts/demo-video.sh 1  # prueba rápida: si muestra la tabla, está todo OK
```

En Windows usa **Git Bash** para los `.sh`. Si no tienes `jq`, instálalo con `winget install jqlang.jq`.

3. Deja abiertas estas **pestañas del navegador**, en este orden:
 1. **GitHub**: github.com/camiluck/banco-xyz-eft (el readme con el diagrama)
 2. **Eureka**: <http://localhost:8761>
 3. **Kafka UI**: <http://localhost:8085>
 4. **GitHub** → [batch-service/src/main/java/cl/bancoxyz/batch/comun/FechaParser.java](#)
 5. **GitHub** → [ms-pagos/src/main/java/cl/bancoxyz/pagos/servicio/PagoService.java](#)
 6. **GitHub** → pestaña **Actions** (la ejecución en verde)
4. Terminal con **letra grande** (Ctrl + + varias veces) y ventana maximizada.
5. En Kaltura, graba **pantalla + webcam** (la cámara en una esquina).

Cómo leer este guion. 🎬 **MOSTRAR** es lo que debe verse en pantalla. 🗣️ **DECIR** es el texto para leer o parafrasear con tus palabras. 💻 **EJECUTAR** es el comando. El script `demo-video.sh` muestra cada comando, lo corre y espera ENTER, así que no tienes que tipear nada en vivo.

1. Presentación (0:00 – 0:20)

🎬 **MOSTRAR:** pantalla completa con la cámara, o el readme de GitHub de fondo.

🗣️ **DECIR:**

"Hola, mi nombre es _____. Les presento mi Evaluación Final Transversal de Desarrollo Backend III: la modernización del backend del Banco XYZ, migrando su sistema legacy a una arquitectura de microservicios con Spring Cloud y Spring Batch."

2. Resumen ejecutivo (0:20 – 1:20)

 **MOSTRAR:** pestaña 1 (GitHub, readme). Baja lentamente hasta el **diagrama de arquitectura** y, al mencionarlas, señala con el mouse cada capa: canales, BFF, gateway, microservicios, Kafka, base de datos.

 **DECIR:**

"El Banco XYZ funcionaba hace más de 30 años sobre COBOL y scripts Shell en un mainframe. Eso le generaba tres problemas: no podía escalar, el mantenimiento era muy caro y era muy difícil integrar canales nuevos.

El objetivo del proyecto fue migrar a una arquitectura de microservicios en la nube. Identifiqué cinco procesos clave: migrar los procesos batch a Spring Batch, dividir el monolito en microservicios, implementar el patrón Backend for Frontend, agregar seguridad distribuida con OAuth2 e integrar mensajería asíncrona con Kafka.

Esto responde a tres requerimientos del negocio: continuidad operativa, o sea que una falla parcial no detenga al banco; escalar a bajo costo; y seguridad e integridad de los datos financieros.

En el diagrama se ve la solución: tres canales, web, móvil y cajeros, cada uno con su propio BFF. Un API Gateway reparte las peticiones entre los microservicios de cuentas, pagos y clientes, que se comunican por eventos en Kafka. Todo se configura con Config Server, se registra en Eureka, se protege con un servidor OAuth2 y corre en Docker."

3. Resultados y comparación con el legacy (1:20 – 4:00)

3.1 Infraestructura (1:20 – 1:40)

 **EJECUTAR:** `./scripts/demo-video.sh 1`

 **MOSTRAR:** la tabla de contenedores, todos *healthy*. Después cambia a la pestaña 2 (**Eureka**) y señala que **MS-CUENTAS** y **MS-PAGOS** tienen **2 instancias** cada uno.

 **DECIR:**

"Aquí está el sistema corriendo: 16 contenedores. En Eureka se ve que cuentas y pagos tienen dos réplicas cada uno. Eso es escalabilidad horizontal: el gateway balancea la carga entre ellas, y puedo agregar más réplicas con un solo comando, sin tocar el código."

3.2 Procesos batch (1:40 – 2:40)

 **EJECUTAR:** `./scripts/demo-video.sh 2` y presiona ENTER entre pantallas.

 **MOSTRAR, en orden:**

1. El job lanzado y los puntitos de "Procesando".
2. **Pasos y particiones:** señala las columnas *leídos*, *escritos* y *omitidos*, y la salida `COMPLETADO_CON_OMISIONES`.
3. **Rechazos agrupados por motivo** (MONTO_VACIO, FECHA_INVALIDA, etc.) y el detalle con línea y contenido.
4. El **resumen diario**.
5. La **falla simulada:** las líneas `[SIMULACION] Falla transitoria`, los rollbacks y el estado final `COMPLETED`.

 **DECIR:**

"Esta es la Parte 1. Migré los tres procesos batch: transacciones diarias, intereses mensuales y estados de cuenta anuales. Cada uno lee los archivos del legacy, valida, normaliza y escribe en chunks transaccionales.

Los datos vienen con errores a propósito: cuatro formatos de fecha, montos vacíos o negativos, duplicados. El sistema legacy fallaba en silencio. Acá cada registro rechazado queda auditado con su línea y su motivo, y el job termina con el estado 'completado con omisiones'.

Además, cada archivo es una partición y se procesan en paralelo.

Ahora simulo que se cae la conexión a la base de datos en medio del proceso. Spring Batch reintenta con espera exponencial y el job termina bien, sin duplicar datos. Si la falla es crítica, el job se reinicia automáticamente desde el último commit."

 **MOSTRAR (5 segundos):** pestaña 1, sección "**Equivalencia con el legacy**" del readme (la tabla).

 **DECIR:**

"Para asegurar que los resultados son equivalentes al legacy, implementé las mismas reglas de forma independiente y las pruebas automáticas comparan los totales: coinciden exactamente."

3.3 Backend for Frontend (2:40 – 3:10)

 **EJECUTAR:** `./scripts/demo-video.sh 3`

 **MOSTRAR:** la respuesta del móvil (corta, con la cuenta enmascarada `****0001`), luego el resumen web y, al final, la **comparación de bytes** WEB vs MOVIL.

 **DECIR:**

"Parte 2: el patrón Backend for Frontend. Antes, todos los canales recibían la misma respuesta pesada del monolito. Ahora el BFF móvil entrega sólo lo esencial: nombres cortos, número de cuenta enmascarado, compresión gzip y ETag. El BFF web entrega el dashboard completo y arma las consultas en paralelo. Miren la diferencia de tamaño entre las dos respuestas."

3.4 Seguridad (3:10 – 3:35)

 **EJECUTAR:** `./scripts/demo-video.sh 4`

 **MOSTRAR:** el `HTTP 401`, el `TERMINAL_NO_AUTORIZADO`, el `PIN_INCORRECTO` y los **dos retiros con el mismo comprobante**.

 **DECIR:**

"Cada canal tiene su propia autenticación con OAuth2 y JWT. Un token del canal web no sirve en la app móvil: responde 401. El cajero, además del token, exige un terminal registrado y el PIN, que se bloquea a los tres intentos fallidos. Todo viaja por HTTPS.

Y si el cajero pierde la conexión y reenvía el retiro, gracias a la clave de idempotencia no se cobra dos veces: mismo comprobante, mismo saldo."

3.5 Kafka (3:35 – 3:50)

 **EJECUTAR:** `./scripts/demo-video.sh 5`

 **MOSTRAR:** la transferencia `COMPLETADO` y el aviso que recibió el cliente 2. Después ve a la pestaña 3 (**Kafka UI**) → *Topics* → `transacciones-completadas` → *Messages* y abre el último mensaje.

 **DECIR:**

"Parte 3: los microservicios se comunican por eventos. Al completar la transferencia, pagos publica un evento en Kafka y el microservicio de clientes lo consume en tiempo real para notificar a la persona que recibió el dinero. Aquí se ve el mensaje en el tópico."

3.6 Comparación con el legacy (3:50 – 4:05)

 **MOSTRAR:** pestaña 1, tabla **"Comparación con el sistema legacy"** del readme.

 **DECIR:**

"En resumen, frente al legacy: pasamos de escalar en vertical a escalar en horizontal; de un sistema donde un error lo botaba todo, a servicios aislados con circuit breaker; de procesos batch que había que repetir desde cero, a reinicios desde el último punto; y de seguridad perimetral, a un token validado en cada servicio y por canal."

4. Desafíos y soluciones (4:05 – 5:35)

Desafío 1: datos legacy con errores (4:05 – 4:30)

 **MOSTRAR:** pestaña 4 (**FechaParser.java**) y señala la lista de formatos de fecha.

 **DECIR:**

"El primer desafío fueron los datos legacy. Las fechas venían en cuatro formatos distintos, había montos vacíos, negativos y registros duplicados. La solución fue leer todo como texto y separar tres casos: lo que se puede corregir, como las fechas, se normaliza; lo sospechoso, como un monto negativo, se guarda marcado como anomalía; y lo inválido se omite, pero queda auditado."

Desafío 2: transferencias entre microservicios (4:30 – 5:00)

 **MOSTRAR:** pestaña 5 (**PagoService.java**) y baja hasta el método `ejecutar` y el método `compensar`.

 **DECIR:**

"El segundo desafío: una transferencia toca dos cuentas, pero ya no hay una sola base de datos. Lo resolví con el patrón Saga: primero se debita, luego se acredita, y si el destino rechaza, se ejecuta una compensación que devuelve el dinero. Cada movimiento lleva una referencia única, así que un reintento nunca descuenta dos veces. Si el servicio de cuentas está caído, el pago queda pendiente y se completa solo cuando el servicio vuelve."

Desafío 3: resiliencia, en vivo (5:00 – 5:35)

 **EJECUTAR:** `./scripts/demo-video.sh 6`

 **MOSTRAR:** `docker compose stop ms-clientes`, luego el **503 SERVICIO_NO_DISPONIBLE** en web, el `datosParciales: true` en móvil y la transferencia que **sí funciona**.

 **DECIR:**

"El tercer desafío fue que una falla no botara todo el sistema. Voy a apagar en vivo el microservicio de clientes. La web responde de inmediato con un error controlado, no con un timeout, gracias al circuit breaker de Resilience4j. La app móvil sigue funcionando con datos parciales. Y las transferencias siguen operando, porque no dependen de ese servicio. Cuando el servicio vuelve, el circuito se cierra solo."

5. Propuestas de mejora y próximos pasos (5:35 – 6:15)

 **MOSTRAR:** pestaña 6 (**GitHub Actions** en verde). Al hablar del despliegue, abre `despliegue.md` en GitHub.

 **DECIR:**

"El proyecto tiene integración continua: en cada cambio, GitHub compila, ejecuta las pruebas y levanta el sistema completo con Docker para probarlo de punta a punta.

Como próximos pasos propongo: primero, el patrón Outbox, para garantizar que los eventos se publiquen aunque Kafka esté caído en ese momento. Segundo, integrar un proveedor de identidad corporativo con autenticación de dos factores y rotación de llaves. Tercero, limitar la cantidad de peticiones por cliente en el gateway. Cuarto, observabilidad con Grafana y Prometheus. Y quinto, desplegar en AWS con ECS Fargate, RDS y MSK, con autoescalado, como dejé documentado en `despliegue.md`."

6. Cierre (6:15 – 6:30)

 **MOSTRAR:** cámara, o el readme.

 **DECIR:**

"El código, las instrucciones para ejecutar cada componente, la guía de despliegue y el informe técnico están en el repositorio. Muchas gracias."

Checklist rápido de la pauta (para no olvidar nada)

Punto que pide la pauta	Dónde está en el video
Resumen ejecutivo: objetivos, alcance, características	Sección 2
Resultados obtenidos y comparación con el legacy	Sección 3 (3.1 a 3.6)

Punto que pide la pauta	Dónde está en el video
Desafíos enfrentados y soluciones implementadas	Sección 4
Propuestas de mejora y próximos pasos	Sección 5
Evidencias del trabajo	Terminal, Eureka, Kafka UI, código y GitHub Actions
Webcam	Todo el video
Duración de 5 a 7 minutos	Unos 6:30

Si algo sale mal durante la grabación

Problema	Solución
Un comando muestra <code>503</code> justo al empezar	Espera 1 minuto (los servicios se están registrando) y repite la escena: <code>./scripts/demo-video.sh N</code>
"No se pudo obtener token"	<code>docker compose ps</code> : algún servicio no está <i>healthy</i> . Ejecuta <code>docker compose up -d</code> y espera
Te equivocaste hablando	No pares: Kaltura permite recortar, o graba por secciones y únelas
Te pasaste de 7 minutos	Acorta la escena 3.2 (muestra sólo los rechazos y la falla simulada)